

Self-Test Documentation — pqc_crypto_module v0.1.0

Disclaimer: This module is prepared for FIPS 140-3 evaluation and is not currently validated.

1. Overview

The module runs Known Answer Tests (KATs) and a continuous RNG health check during initialization. All tests must pass before the module transitions to Approved state and makes cryptographic services available.

Self-tests are implemented in `src/self_tests.rs` and executed by `self_tests::run_all()`, which is called from `api::initialize_approved_mode()`.

2. Test Inventory

#	Test Name	Algorithm	Standard	Type
1	kat_sha3_256	SHA3-256	FIPS 202	Known Answer Test
2	kat_mldsa65	ML-DSA-65	FIPS 204	Known Answer Test (sign/verify/reject)
3	kat_mlkem	ML-KEM-768	FIPS 203	Known Answer Test (keygen/encaps/decaps)
4	test_rng	OsRng	SP 800-90B	Continuous random number generator test

Tests execute sequentially. If any test fails, execution stops and the remaining tests are skipped.

3. Test Descriptions

3.1 SHA3-256 KAT (`kat_sha3_256`)

Purpose: Verify the SHA3-256 implementation produces correct output.

Procedure: 1. Compute `SHA3-256("")` (empty string). 2. Compare the hex-encoded digest against the known value:
`a7ffc6f8bf1ed76651c14756a061d662f580ff4de43b49fa82d80a4b80f8434a` 3. Compute `SHA3-256("")` a second time. 4. Verify both digests are identical (determinism check).

Failure condition: Digest mismatch or non-deterministic output.

Error message: "SHA3-256 KAT: empty string digest mismatch" or "SHA3-256 KAT: non-deterministic".

3.2 ML-DSA-65 KAT (`kat_mldsa65`)

Purpose: Verify ML-DSA-65 key generation, signing, and verification.

Procedure: 1. Generate a fresh ML-DSA-65 keypair using `generate_keypair_raw()`. 2. Sign the message `b"FIPS-204-KAT-ML-DSA-65"` with the private key. 3. Verify the signature against the public key and original message. **Must succeed.** 4. Corrupt the signature (flip bits in first byte: `sig[0] ^= 0xff`). 5. Verify the corrupted signature. **Must fail.** 6. Verify the original signature against a different message (`b"wrong"`). **Must fail.**

Failure conditions: - Sign operation fails. - Valid signature is rejected. - Corrupted signature is accepted. - Wrong message verifies.

Error messages: "ML-DSA sign: ...", "ML-DSA KAT: sign-then-verify failed", "ML-DSA KAT: corrupted signature was accepted", "ML-DSA KAT: wrong message verified".

3.3 ML-KEM KAT (`kat_mlkem`)

Purpose: Verify ML-KEM-768 key generation, encapsulation, and decapsulation execute without error.

Procedure: 1. Generate a fresh ML-KEM-768 keypair using `kem_keygen_raw()`. 2. Encapsulate with the public key, producing a ciphertext and shared secret (`ss1`). 3. Decapsulate the ciphertext with the private key, producing shared secret (`ss2`). 4. Verify `ss1 == ss2` (shared secret roundtrip). 5. Attempt decapsulation with an invalid ciphertext (1088 bytes of `0xAA`). 6. Verify invalid ciphertext produces a different shared secret or an error (implicit rejection).

Failure conditions: Keygen, encapsulate, or decapsulate error; shared secrets do not match; corrupted ciphertext produces same shared secret.

Error messages: "ML-KEM keygen: ...", "ML-KEM encaps: ...", "ML-KEM decaps: ...", "ML-KEM KAT: shared secrets from encapsulate and decapsulate do not match", "ML-KEM KAT: corrupted ciphertext produced same shared secret".

3.4 Continuous RNG Test (`test_rng`)

Purpose: Verify the CSPRNG produces non-repeating output (NIST SP 800-90B Section 4.3 alignment).

Procedure: 1. Generate 32 bytes of random data (sample A). 2. Generate 32 bytes of random data (sample B). 3. Compare A and B. **Must differ.**

Failure condition: Both 32-byte samples are identical.

Error message: "continuous RNG test failed: repeated output".

4. When Self-Tests Run

Trigger	Tests Run
<code>api::initialize_approved_mode()</code>	All 4 tests (power-up self-tests)
Process startup (via application calling <code>initialize_approved_mode()</code>)	All 4 tests
Module re-initialization	Not supported. <code>initialize_approved_mode()</code> can be called again, but if the module is already in Error state, self-tests will run and the result determines the new state.

Self-tests do **not** run: - On individual cryptographic operations (no conditional self-tests currently implemented). - Periodically in the background. - On key generation (pair-wise consistency test is not yet implemented).

5. Failure Behavior

When any self-test fails:

- `self_tests::run_all()` returns `Err(CryptoError::SelfTestFailed(description))`.
- `api::initialize_approved_mode()` sets the module state to Error.
- The error is returned to the caller.
- All subsequent calls to any approved API function return `Err(CryptoError::ModuleInErrorState)`.
- All subsequent calls to legacy API functions also fail (the `ensure_not_approved()` guard passes, but the overall module is effectively unusable since Error maps to a non-Approved state).

6. The module **cannot recover**. The process must be restarted.

This is fail-closed behavior: no cryptographic service is available after a self-test failure.

6. Internal vs. Public Functions

Self-tests use `*_raw` internal functions that bypass the `require_approved()` guard:

Internal function	Used by	Why
<code>sha3_256_raw(data)</code>	<code>kat_sha3_256</code>	Module is in SelfTesting state, not yet Approved
<code>generate_keypair_raw()</code>	<code>kat_mldsa65</code>	Same
<code>sign_message_raw(sk, msg)</code>	<code>kat_mldsa65</code>	Same
<code>verify_signature_raw(pk, msg, sig)</code>	<code>kat_mldsa65</code>	Same
<code>kem_keygen_raw()</code>	<code>kat_mlkem</code>	Same
<code>encapsulate_raw(pk)</code>	<code>kat_mlkem</code>	Same
<code>decapsulate_raw(sk, ct)</code>	<code>kat_mlkem</code>	Same
<code>continuous_rng_test()</code>	<code>test_rng</code>	RNG test needs to run before Approved

These functions have `pub(crate)` visibility and are not accessible outside the crate.

7. Future Enhancements

1. **Pair-wise consistency test:** Add a key-pair consistency test after ML-DSA key generation (sign and verify with freshly generated pair).
2. **ML-KEM shared secret equality:** Verify that encapsulate and decapsulate produce the same shared secret once the real FIPS 203 implementation is available.
3. **Conditional self-tests:** Run algorithm-specific tests on first use of each algorithm, not only at startup.
4. **Periodic self-tests:** Optionally re-run KATs at configurable intervals during long-running processes.